

CSE 450 CAPSTONE PROJECT · DEPARTMENT OF CSE, BUET

Exploratory Testing Automation



An autonomous agent that decides what to test, runs it on a real Android device, and learns from every run.

TEAM

Mohammad Raihan Rashid · 2105046
Mehemud Azad · 2105014
Jonayed Mohiuddin Shanto · 2105060
Khalid Hasan Tuhin · 2105002
Niloy Kumar Mondal · 2105044

SUPERVISOR

Dr. A. B. M. Alim Al Islam
Professor, Department of CSE, BUET

INDUSTRY PARTNER

Samsung R&D Institute Bangladesh

THE PROBLEM

Why exploratory testing is hard to automate

Manual exploratory testing

Finds real bugs, because a person knows what correct looks like.

But it does not scale, and the knowledge leaves with the tester.

Scripted automation

Repeatable, and cheap to run again.

But it only checks what its author thought to check. It confirms; it does not discover.

Random (Monkey) testing

Explores at scale, with no setup.

But it has no idea what correct looks like. It can report a crash, not a wrong result.

“An automated tool can tell you an application crashed. It cannot tell you it quietly charged the wrong price, because it has no idea what the right price idea what the right price was.”

RELATED WORK

What existing research agents miss

Capability	Research agents today	Our system
Reads the requirements specification	No, so no oracle beyond a crash	Yes: every requirement becomes a queryable node
Remembers across runs	Only within one session	A persistent knowledge graph
Separates its own failures from app defects	No	Three-way fault attribution
Chooses its own test objectives	5 of 7 are handed a task	Yes, and each test cites a requirement

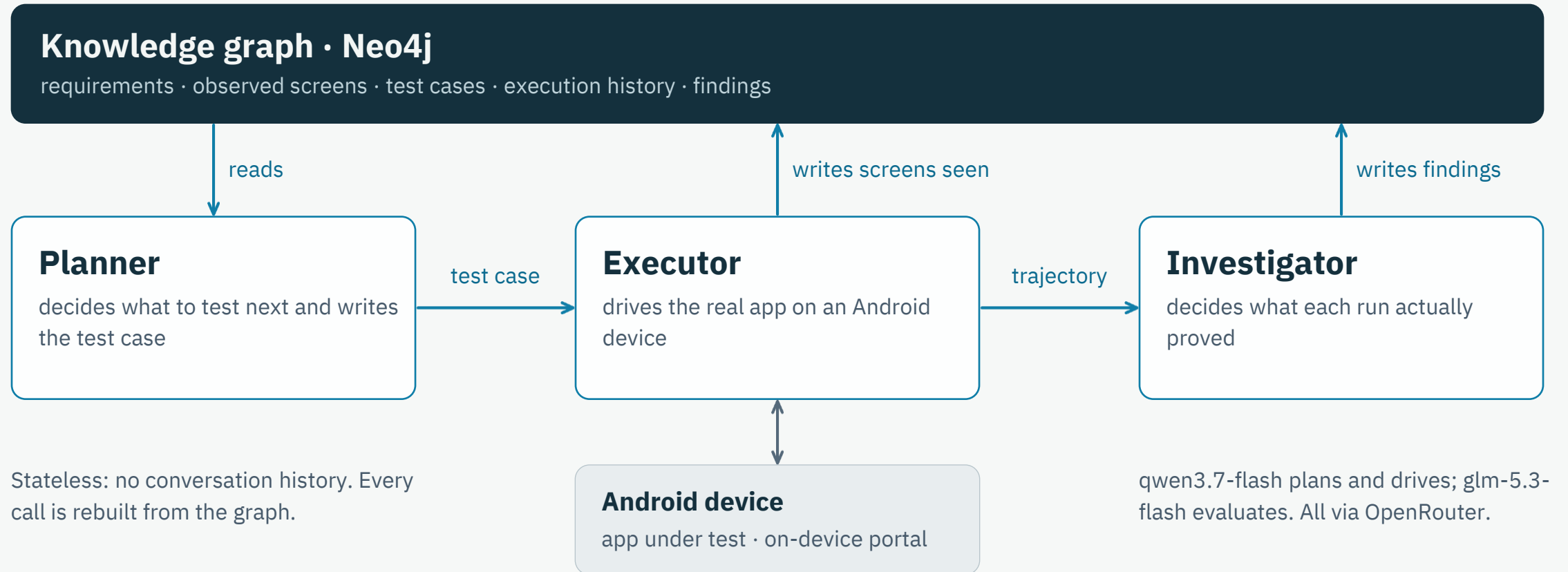
From our review of seven LLM-based GUI testing systems, among them GPTDroid, DroidAgent and AutoDroid.

THE PARTNER'S BRIEF · SAMSUNG R&D INSTITUTE BANGLADESH

**“Every execution cycle must make
make the agent smarter than the
the last.”**

Our answer: three stateless AI agents that never talk to each other. Everything they learn goes into one shared knowledge graph, and every new test is planned from it.

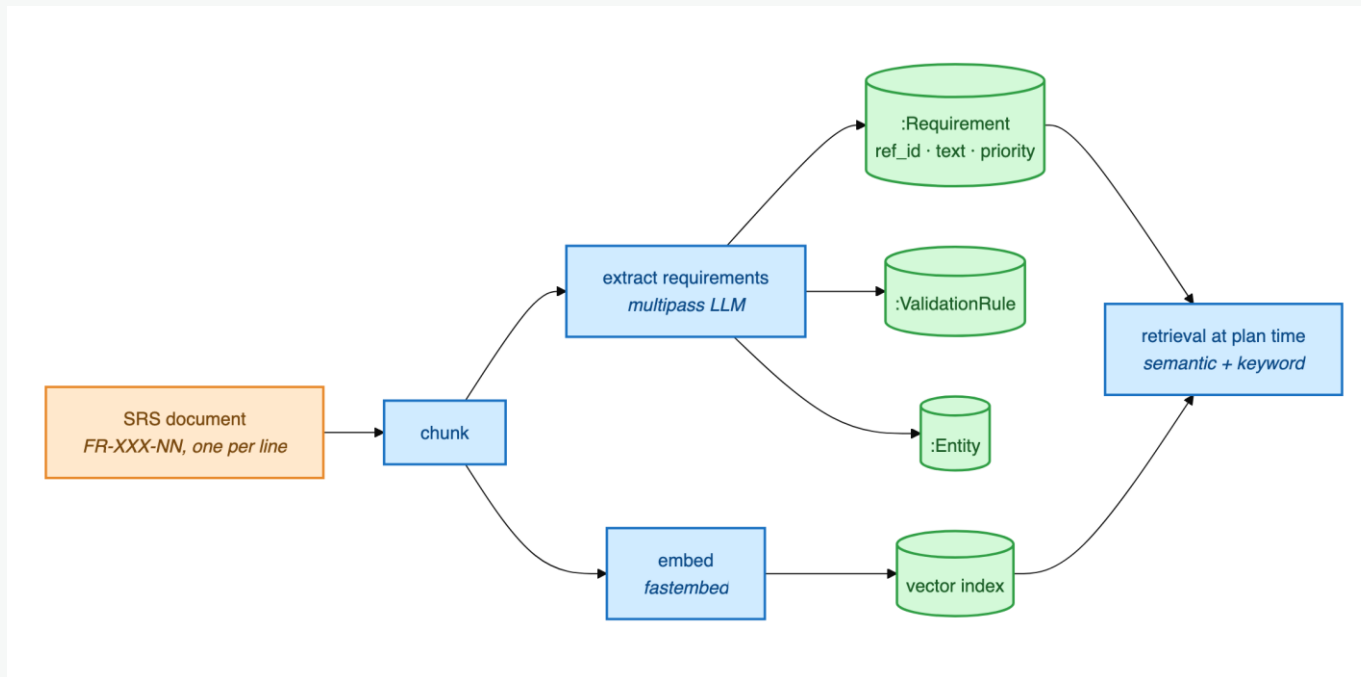
Three agents, one shared memory



INGESTION

A document becomes queryable requirements

Prose in. Individually identified, traceable requirements out.



Each requirement carries

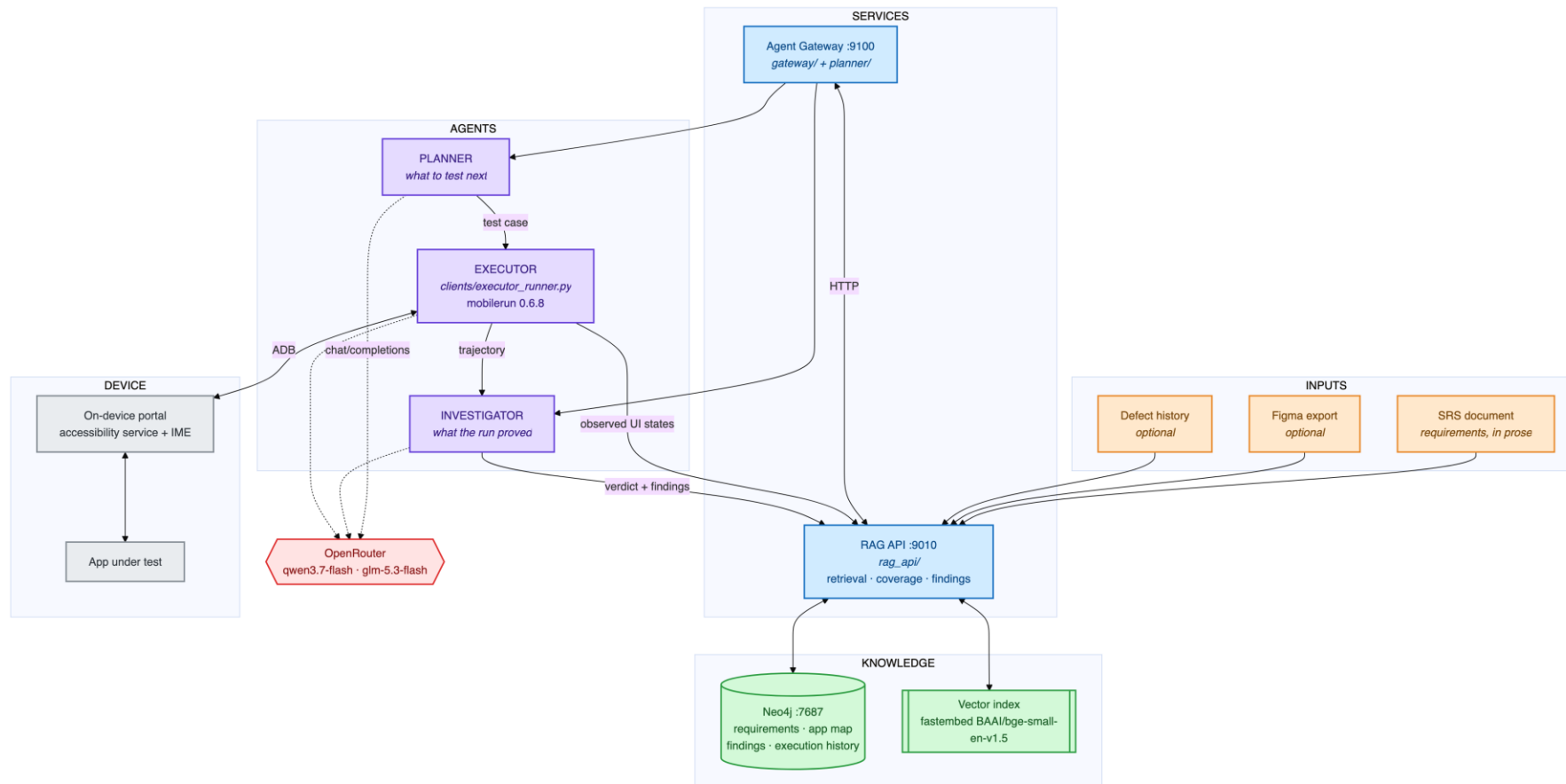
id · type · priority
actor · action
constraints
acceptance
feature area

plus validation rules and the domain entities
they act on

*All three documents are optional. A target with
none of them still runs.*

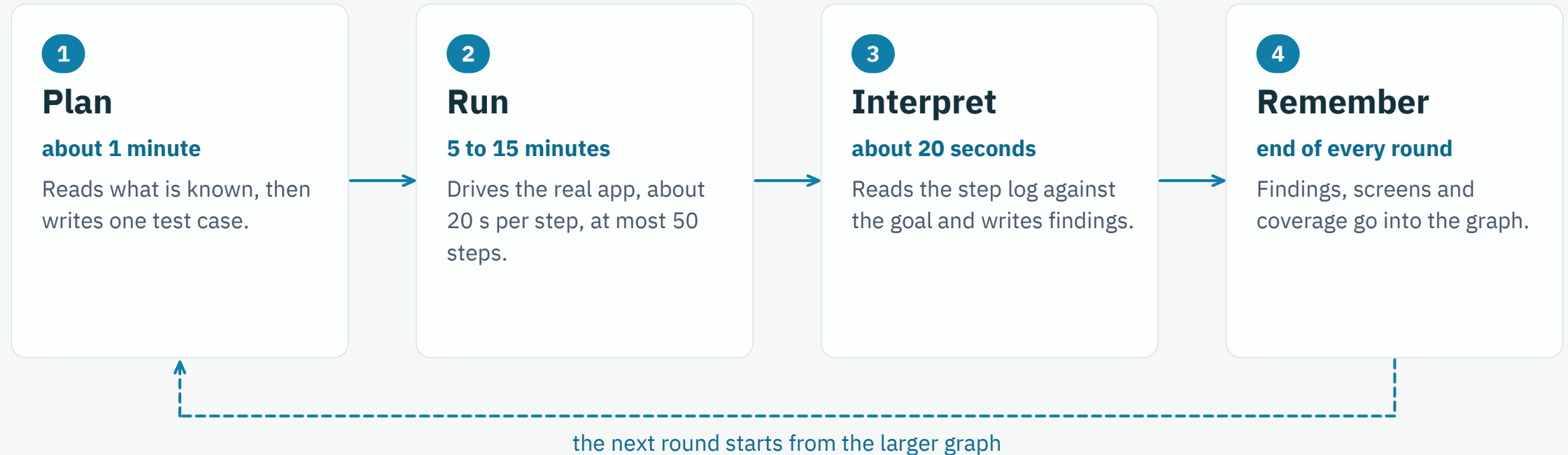
Each requirement links back to the exact chunk of text it came from, which is what makes requirement coverage reportable.

Every component, and how they connect



HOW A TEST RUNS

One round: plan, run, interpret, remember



Every model call is stateless. The agent improves only because the graph grew.

The fourteen messages of one round

1–5 · Planner

proposes and stores the test

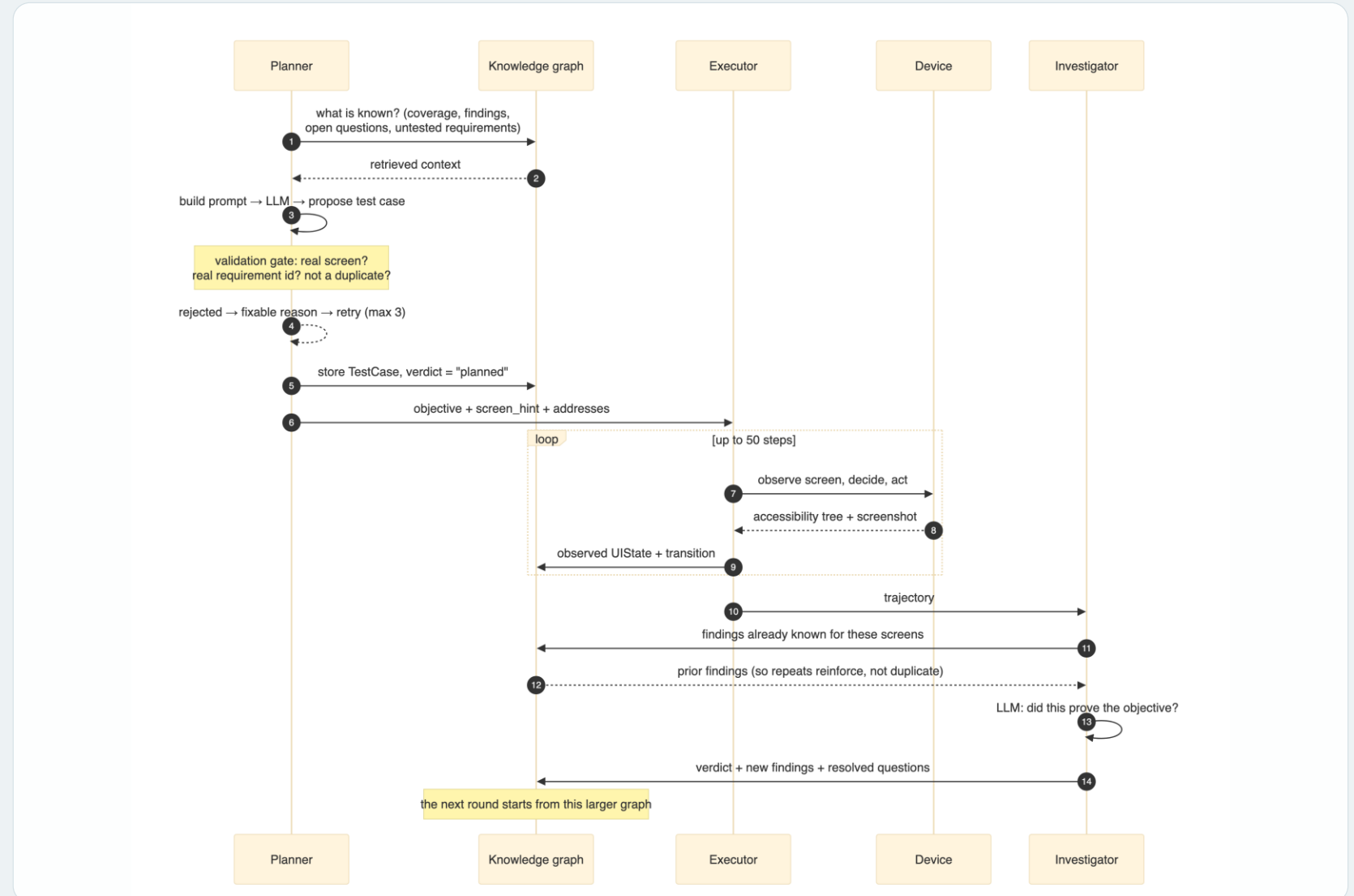
6–9 · Executor

runs it, posting every screen

10–14 · Investigator

judges it, writes findings

Only the graph carries over.



MODELS

Which model does what, and why

Role	Model	Why this choice
Planner	qwen/qwen3.7-flash	Long context and dependable structured output matter more than deep reasoning. About 8% of a test's cost.
Executor (Android)	qwen/qwen3.7-flash, with vision	Reads the accessibility tree and a screenshot on every step, so it is about 92% of a test's cost.
Investigator and rate-limit fallback	z-ai/glm-5.3-flash	A different model family, so no model grades its own execution. Low reasoning effort cut an evaluation from 113.8 s to 22.7 s.
Specification extraction	qwen/qwen3-235b-a22b	Run once per document, and every later decision rests on it, so accuracy is worth paying for here.
Embeddings	BAAI/bge-small-en-v1.5	Runs locally through fastembed, with no per-call charge.

All routed through OpenRouter. Every campaign in this deck ran on the planner, executor and investigator above; the planner is next moving to deepseek-v4-flash.